

Pocket Synth — Documento de diseño de la primera iteración

Proyecto: Pocket Synth para M5Stack Cardputer ADV

Iteración: 1 — Una sola voz polifónica, sin ADSR, sin LFO

Fecha: 2026-06-18

Revisión: v1.2 - nombre pocketsynth e iconos compactos de forma de onda

Resolución objetivo: 240 × 135 px, orientación landscape

UI de referencia: cardputer-pocket-synth-single-voice-compact-8poly-fnwave-icons-v5.json

1. Objetivo de la primera iteración

El objetivo de esta primera iteración es construir un sintetizador real en tiempo real sobre el Cardputer ADV, generando las notas matemáticamente en el propio dispositivo y enviando audio al sistema de salida.

Esta versión no busca todavía sonar como un sintetizador completo con modulación, envolventes o capas complejas. Busca validar la base crítica:

Teclado físico

- > eventos note on/off
- > motor de síntesis
- > osciladores matemáticos
- > mezcla de hasta 8 notas
- > volumen maestro
- > buffer de audio
- > salida I2S / speaker

La primera iteración debe demostrar que el Cardputer puede funcionar como motor de síntesis real-time estable.

2. Alcance funcional

Incluido en la Iteración 1

- Una sola voz/instrumento.
- Polifonía máxima de 8 notas simultáneas.
- Generación matemática de formas de onda.
- Cuatro formas de onda seleccionables:
 - Senoidal.
 - Cuadrada.
 - Rectangular con duty fijo.
 - Diente de sierra.
- Lectura del teclado del Cardputer.
- Mapeo de teclas físicas a notas musicales.
- Cambio de forma de onda mediante combinaciones Fn + 1, Fn + 2, Fn + 3, Fn + 4.
- Volumen maestro.
- Feedback visual de teclas pulsadas.
- Indicador de polifonía actual.
- Detección básica de acordes.
- UI compacta en una sola pantalla.
- Nombre de la aplicación mostrado en pequeño como pocketsynth en la zona superior.
- Iconos pequeños de forma de onda junto a los selectores Fn + 1..4.

Excluido de la Iteración 1

- ADSR.

- LFO.
- Filtros.
- Múltiples canales/voices de mezcla.
- Presets.
- Efectos.
- Secuenciador.
- Grabación.
- Arpegiador.
- Menús profundos.
- Edición avanzada de parámetros.

La decisión de excluir estas funcionalidades no es una limitación del diseño, sino una forma de proteger la base real-time del sintetizador.

3. Concepto sonoro de la primera iteración

Aunque hablamos de “una sola voz”, en esta iteración la palabra voz significa **un único instrumento o capa sonora**.

Dentro de esa voz puede haber hasta 8 notas simultáneas. Es decir:

```
Voice 1
├─ Nota activa 1
├─ Nota activa 2
├─ Nota activa 3
├─ ...
└─ Nota activa 8
```

Cada nota activa mantiene su propio estado de fase.

Esto es importante porque no se puede generar todo con una única variable global de tiempo. Cada nota necesita avanzar su fase según su frecuencia.

4. Modelo matemático de síntesis

4.1 Fase por nota

Cada nota activa tendrá:

```
struct ActiveNote {
    bool active;
    float frequency;
    float phase;
    float phaseIncrement;
};
```

La fase se expresa normalmente en rango normalizado:

```
phase = 0.0 ... 1.0
```

Cada muestra de audio avanza la fase:

```
phase += phaseIncrement;

if (phase >= 1.0f) {
    phase -= 1.0f;
}
```

Donde:

```
phaseIncrement = frequency / sampleRate;
```

Ejemplo:

A4 = 440 Hz

sampleRate = 22050 Hz

phaseIncrement = 440 / 22050 = 0.01995 aprox.

5. Formas de onda

5.1 Senoidal

Primera versión sencilla:

```
sample = sinf(phase * 2.0f * PI);
```

Más adelante se podrá sustituir por tabla precalculada para ahorrar CPU.

5.2 Cuadrada

```
sample = phase < 0.5f ? 1.0f : -1.0f;
```

5.3 Rectangular

La rectangular será una onda tipo pulso con duty fijo.

```
sample = phase < pulseWidth ? 1.0f : -1.0f;
```

Valor inicial recomendado:

```
pulseWidth = 0.25f;
```

5.4 Diente de sierra

```
sample = 2.0f * phase - 1.0f;
```

6. Mezcla y normalización

Al sumar varias notas, la amplitud puede superar el rango permitido.

Rango interno recomendado:

```
-1.0 ... +1.0
```

Si se suman 4 notas con amplitud alta, la señal podría saturar.

Por eso la iteración 1 usará una estrategia simple de gain staging:

```
mixed = sum(activeNotes)
```

```
mixed *= perNoteGain
```

```
mixed /= sqrt(activeNoteCount)
```

```
mixed *= masterVolume
```

```
mixed = clamp(mixed, -1.0f, 1.0f)
```

Valores iniciales recomendados:

```
constexpr int MAX_POLYPHONY = 8;
constexpr float PER_NOTE_GAIN = 0.45f;
float masterVolume = 0.70f;
```

Render conceptual:

```
float renderSample(SynthState& state) {
    float mixed = 0.0f;
    int activeCount = 0;

    for (auto& note : state.notes) {
        if (!note.active) continue;

        float s = oscillatorSample(note.phase, state.waveform);

        mixed += s * PER_NOTE_GAIN;

        note.phase += note.phaseIncrement;
        if (note.phase >= 1.0f) {
            note.phase -= 1.0f;
        }

        activeCount++;
    }

    if (activeCount > 1) {
        mixed /= sqrtf((float)activeCount);
    }

    mixed *= state.masterVolume;

    if (mixed > 1.0f) mixed = 1.0f;
    if (mixed < -1.0f) mixed = -1.0f;

    return mixed;
}
```

7. Frecuencia de muestreo y buffer

Valores recomendados para comenzar:

```
constexpr int SAMPLE_RATE = 22050;
constexpr int AUDIO_BUFFER_FRAMES = 128;
constexpr int MAX_POLYPHONY = 8;
```

Motivo:

- 22050 Hz reduce carga de CPU.
- 128 frames da una latencia razonable y margen de estabilidad.
- 8 notas es suficiente para tocar acordes complejos en el teclado del Cardputer.

Cuando todo esté estable se podrá evaluar:

```
SAMPLE_RATE = 32000;
```

pero no debería ser el primer objetivo.

8. Arquitectura FreeRTOS

La regla principal es:

El audio manda.

La tarea de audio nunca debe esperar por la UI, logs, teclado, SD, gráficos o locks largos.

8.1 Tareas propuestas

AudioTask

- Prioridad alta
- Genera buffers de audio
- Escribe a I2S / speaker
- No hace logs
- No reserva memoria dinámica

InputTask

- Prioridad media
- Escanea teclado
- Detecta cambios note on/off
- Envía eventos al motor

SynthControlTask

- Prioridad media
- Procesa eventos
- Actualiza estado del sinte
- Actualiza waveform, volumen y notas activas

UiTask

- Prioridad baja
- Redibuja sólo si hay cambios
- Pinta teclas pulsadas
- Actualiza poly, waveform, volumen y acorde

ChordTask

- Opcional
- Prioridad baja/media
- Recalcula acorde sólo cuando cambia el conjunto de notas pulsadas

En una primera versión, ChordTask puede estar integrada dentro de SynthControlTask.

9. Comunicación entre tareas

9.1 Eventos de entrada

```
enum class EventType {
    NoteOn,
    NoteOff,
    SetWaveform,
    SetVolume
};

struct SynthEvent {
    EventType type;
    uint8_t key;
    uint8_t noteIndex;
```

```
float value;
};
```

InputTask envía eventos:

```
xQueueSend(synthEventQueue, &event, 0);
```

SynthControlTask consume eventos y actualiza el estado.

9.2 Estado del sintetizador

```
enum class Waveform {
    Sine,
    Square,
    Rectangle,
    Saw
};

struct SynthState {
    Waveform waveform;
    float masterVolume;
    ActiveNote notes[MAX_POLYPHONY];
    uint32_t pressedMask;
};
```

Para la Iteración 1 se puede empezar copiando un estado pequeño con sección crítica corta.

Más adelante, si hiciera falta, se puede pasar a doble buffer de estado:

```
controlState
audioState
```

El control escribe en un estado y el audio lee otro.

10. Reglas críticas del AudioTask

Dentro del camino de audio no debe haber:

```
printf(...)
ESP_LOGI(...)
malloc(...)
new
std::vector dinámico
String
lectura de SD
redibujado de pantalla
espera de mutex largo
operaciones I2C lentas
```

El bucle de audio debe ser predecible:

```
while (true) {
    renderAudioBuffer(buffer, AUDIO_BUFFER_FRAMES);
    i2s_write(...);
}
```

11. Mapeo de controles

11.1 Teclas de piano

Teclas blancas:

z x c v b n m q w e r t y u i

Teclas negras:

s d g h j 2 3 5 6 7

Mapa inicial:

Tecla	Nota
z	C4
s	C#4 / Db4
x	D4
d	D#4 / Eb4
c	E4
v	F4
g	F#4 / Gb4
b	G4
h	G#4 / Ab4
n	A4
j	A#4 / Bb4
m	B4
q	C5
2	C#5 / Db5
w	D5
3	D#5 / Eb5
e	E5
r	F5
5	F#5 / Gb5
t	G5
6	G#5 / Ab5
y	A5
7	A#5 / Bb5
u	B5
i	C6

11.2 Formas de onda

Las formas de onda se cambian usando la tecla Fn como modificador. Esto evita conflictos con el piano, especialmente porque algunas teclas numéricas también forman parte del teclado musical.

Combinación	Forma de onda
Fn + 1	Senoidal
Fn + 2	Cuadrada
Fn + 3	Rectangular
Fn + 4	Diente de sierra

11.3 Volumen

Inicialmente:

Fn + ↑ : subir volumen

Fn + ↓ : bajar volumen

En Cardputer, las flechas pueden estar en capa Fn sobre teclas físicas. La implementación debe usar el mapeo real que exponga la librería de teclado.

12. Diseño de UI de la primera iteración

La UI debe mostrar únicamente lo que existe en esta fase.

No debe mostrar:

- Mixer.
- Múltiples voces.
- ADSR.
- LFO.
- Filtro.
- Presets.

Elementos visibles:

```
pocketsynth
0/8
CHORD: --
VOICE: V1
VOL
WAVE: Fn1 [sine icon] Fn2 [square icon] Fn3 [pulse icon] Fn4 [saw icon]
Wave preview
Output preview
Piano
```

La pantalla debe mantenerse limpia y legible.

12.1 Nombre de la aplicación

La aplicación se llamará oficialmente:

```
pocketsynth
```

El nombre debe mostrarse en pequeño en la parte superior de la pantalla. En esta fase no debe dominar la UI: el protagonismo debe quedar en el estado musical actual, la polifonía, el acorde detectado, la forma de onda y el piano.

Decisión de estilo:

- Texto en minúsculas: pocketsynth.
- Tamaño pequeño.
- Posición superior izquierda o superior central-izquierda.
- Sin ocupar una línea completa si compromete la legibilidad.

12.2 Iconos de forma de onda

Donde aparecen los nombres de las ondas se mostrarán iconos pequeños con la forma de la onda. La intención es que la UI sea reconocible de un vistazo incluso en una pantalla de 240 x 135 px.

Los iconos serán muy simples, de pocos píxeles, y deberán representar:

Control	Onda	Icono esperado
Fn + 1	Senoidal	curva suave tipo seno
Fn + 2	Cuadrada	escalón cuadrado
Fn + 3	Rectangular	pulso estrecho / PWM fijo
Fn + 4	Diente de sierra	rampa ascendente con caída brusca

En la UI puede mantenerse una etiqueta mínima de apoyo, por ejemplo F1, F2, F3, F4, pero la identificación visual principal debe ser el icono.

No se deben usar iconos recargados ni gráficos grandes. Deben funcionar como pequeños pictogramas técnicos.

12.3 Feedback visual

Cuando una tecla esté pulsada:

- Tecla blanca:
 - fill: #d8ecff
 - stroke: #7cc7ff
- Tecla negra:
 - fill: #25415f
 - stroke: #7cc7ff

Este feedback visual debe depender del estado real de notas activas, no sólo del evento de pulsación.

13. Detección de acordes

La detección de acordes no debe bloquear audio.

Entrada:

Conjunto de notas activas

Salida:

Nombre de acorde

Ejemplos:

C
Cm
CMaj7
CMaj7#5/C
Gsus4/D

Primera versión recomendable:

1. Convertir notas activas a pitch classes.
2. Calcular nota grave.
3. Probar patrones básicos:
 - Mayor.
 - Menor.
 - Disminuido.
 - Aumentado.
 - Sus2.
 - Sus4.
 - 7.
 - Maj7.
 - m7.
4. Mostrar inversión si la nota grave no coincide con la raíz.

Formato:

CMaj7/E

si el acorde es Cmaj7 con E como bajo.

14. Fases internas de la Iteración 1

Fase 1A — Audio mínimo

Objetivo:

- Inicializar salida de audio.
- Generar una nota fija.
- Confirmar que el buffer no se corta.

Criterio de aceptación:

Suena una nota continua sin cortes audibles durante al menos 60 segundos.

Fase 1B — Teclado monofónico

Objetivo:

- Leer teclado.
- Mapear una tecla a una nota.
- Generar note on/off.

Criterio de aceptación:

Al pulsar z suena C4.
Al soltar z deja de sonar.
El audio no se bloquea.

Fase 1C — Polifonía de 8 notas

Objetivo:

- Permitir hasta 8 notas simultáneas.
- Mantener una fase independiente por nota.
- Normalizar la suma.

Criterio de aceptación:

Se pueden tocar acordes de 3, 4 y 5 notas sin clipping evidente.
El contador de polifonía muestra n/8.
Si se excede el máximo, el sistema no se rompe.

Política inicial para exceder 8 notas:

Ignorar nuevas notas hasta que se libere alguna activa.

Alternativa futura:

Voice stealing.

Fase 1D — Selección de forma de onda

Objetivo:

- Fn + 1: seno.
- Fn + 2: cuadrada.
- Fn + 3: rectangular.
- Fn + 4: diente de sierra.
- Mostrar iconos pequeños de cada forma de onda en la UI.

Criterio de aceptación:

Se puede cambiar de onda con `Fn + 1`, `Fn + 2`, `Fn + 3` o `Fn + 4` mientras suena una nota o La UI refleja la onda seleccionada mediante estado activo y mediante el icono correspondiente.

Fase 1E — UI compacta

Objetivo:

- Mostrar estado mínimo.
- Pintar teclas activas.
- Mostrar waveform y output preview.
- Mostrar volumen y polifonía.

Criterio de aceptación:

La UI se actualiza sin provocar cortes de audio.
La tasa de refresco puede ser baja, pero debe ser estable.

Recomendación:

15-20 FPS máximo.
Redibujar sólo si hay cambios.

Fase 1F — Detección de acordes

Objetivo:

- Identificar acordes básicos a partir de notas activas.
- Mostrar el nombre del acorde en pantalla.

Criterio de aceptación:

Triadas mayores y menores se detectan correctamente.
Séptimas básicas se detectan correctamente.
Las inversiones muestran bajo cuando procede.

Fase 1G — Estabilización y profiling

Objetivo:

- Medir carga de CPU.
- Comprobar underruns.
- Comprobar latencia.
- Ajustar tamaño de buffer.
- Ajustar sample rate si procede.

Criterio de aceptación:

El sistema permite tocar durante varios minutos sin cortes de audio, bloqueos ni degradación de

15. Decisiones técnicas iniciales

Decisión	Valor
Polifonía	8 notas máximo
Voces/canales	1
ADSR	No
LFO	No
Filtro	No
Sample rate inicial	22050 Hz
Buffer inicial	128 frames
Formato interno	float
Salida final	int16
UI FPS	15-20 FPS
Nombre visible de app	pocketsynth, pequeño en la zona superior
Selectores de onda	Fn + 1..4 con iconos compactos de forma de onda
Normalización	$\sqrt{\text{activeNoteCount}}$ + headroom
Onda rectangular	Duty fijo, inicialmente 25%

16. Riesgos principales

16.1 Cortes de audio

Causa probable:

- AudioTask bloqueado.
- Buffer demasiado pequeño.
- UI demasiado pesada.
- Logs dentro del audio path.

Mitigación:

- Prioridad alta para AudioTask.
- Sin logs en render.
- Sin memoria dinámica.
- UI a baja frecuencia.
- Buffer ajustable.

16.2 Clipping

Causa probable:

- Suma de muchas notas.
- Ganancia por nota demasiado alta.
- Volumen maestro alto.

Mitigación:

- PER_NOTE_GAIN.
- División por $\sqrt{\text{activeNoteCount}}$.
- Clamp final.
- Medidor o indicador futuro de clipping.

16.3 Latencia de teclado

Causa probable:

- InputTask con periodo demasiado lento.
- Debounce excesivo.
- Bloqueos en lectura.

Mitigación:

- Escaneo cada 5-10 ms.
- Eventos sólo en cambios.
- Separar input de UI.

16.4 UI saturada

Causa probable:

- Demasiados datos en 240 × 135.
- Textos largos.
- Gráficas demasiado grandes.

Mitigación:

- UI compacta.
 - Abreviaturas.
 - Mostrar sólo lo implementado.
 - Evitar conceptos futuros en pantalla.
-

17. Criterio global de éxito de la Iteración 1

La iteración se considerará completada cuando:

El Cardputer ADV pueda actuar como sintetizador real-time básico:
una sola voz polifónica de hasta 8 notas, con selección de forma de onda,
volumen maestro, feedback visual de teclas, detección de acorde y audio estable.

Prueba final:

1. Encender Pocket Synth.
 2. Pulsar z, x, c y escuchar notas individuales.
 3. Tocar acordes de 3-5 notas.
 4. Cambiar entre `Fn + 1`, `Fn + 2`, `Fn + 3`, `Fn + 4` mientras suena y ver el selector/icono
 5. Subir y bajar volumen.
 6. Ver contador de polifonía.
 7. Ver acorde detectado.
 8. Mantener uso durante varios minutos sin cortes.
-

18. Roadmap posterior

Una vez cerrada la Iteración 1:

Iteración 2 — Múltiples canales/voices

- Añadir más capas sonoras.
- Cada canal con waveform y volumen propios.
- Mezclador.
- Navegación entre canales.

Iteración 3 — ADSR

- Attack.
- Decay.
- Sustain.
- Release.

- Aplicado por nota.

Iteración 4 — LFO

- Vibrato.
- Tremolo.
- PWM.
- Modulación de parámetros.

Iteración 5 — Filtro

- Low-pass.
- High-pass.
- Band-pass.
- Resonancia básica.

Iteración 6 — Presets

- Guardar y cargar patches.
 - Posible uso de SD.
 - Exportación/importación.
-

19. Principio rector del proyecto

Primero audio estable.
Después interacción.
Después UI.
Después musicalidad.
Después modulación.

Esta primera iteración debe ser pequeña, real, medible y sólida. El objetivo no es tener muchas funciones, sino tener una base de síntesis fiable sobre la que se pueda construir el resto del instrumento.